

Implementation of graph and searching (DFS and BFS)

```
from collections import deque

# Graph using adjacency list
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'E', 'F'],
    'C': ['F'],
    'D': ['A'],
    'E': ['B'],
    'F': ['B', 'C']
}

# DFS
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    print(start, end=" ")

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

# Display graph
print("Graph:")
for vertex in graph:
    print(vertex, "->", graph[vertex])

# Traversals
print("\nDFS Traversal:")
dfs(graph, 'A')
```

Graph:
A -> ['B', 'D']
B -> ['A', 'E', 'F']
C -> ['F']
D -> ['A']
E -> ['B']
F -> ['B', 'C']

DFS Traversal:
A B E F C D

```
from collections import deque

# Graph using adjacency list
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'E', 'F'],
    'C': ['F'],
    'D': ['A'],
    'E': ['B'],
    'F': ['B', 'C']
}

# BFS
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Display graph
print("Graph:")
```

```
for vertex in graph:  
    print(vertex, "->", graph[vertex])
```

```
print("\nBFS Traversal:")  
bfs(graph, 'A')
```

Graph:

```
A -> ['B', 'D']  
B -> ['A', 'E', 'F']  
C -> ['F']  
D -> ['A']  
E -> ['B']  
F -> ['B', 'C']
```

BFS Traversal:

A B D E F C